Processingをはじめよう

第9章

関数

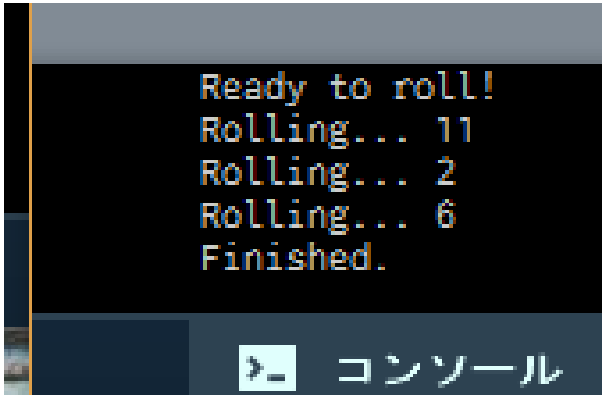
目次

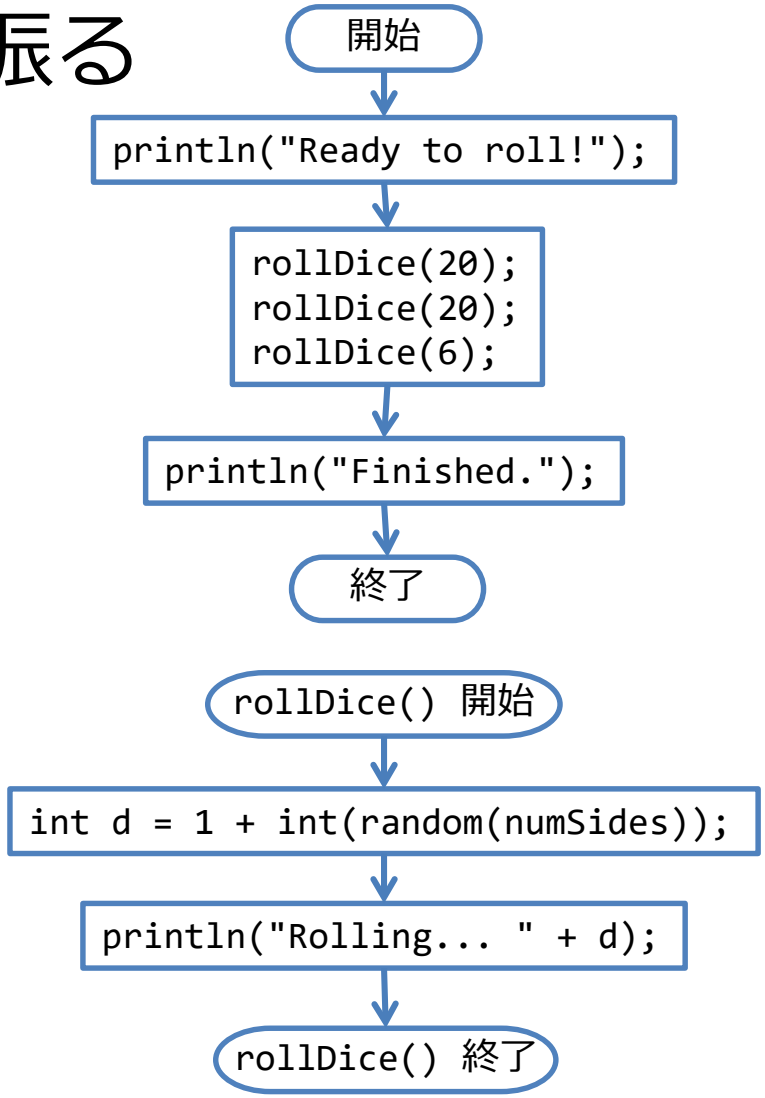
- 関数の基礎
- 関数を作る
- 値を返す
- Robot 7: Functions

関数の基礎

- Example 9-1 サイコロを振る

```
void setup() {
  println("Ready to roll!");
  rollDice(20);
  rollDice(20);
  rollDice(6);
  println("Finished.");
}

void rollDice(int numSides) {
  int d = 1 + int(random(numSides));
  println("Rolling... " + d);
}
```





関数の定義

- 関数の定義

```
戻り値の型　関数名(引数の型 引数名, ...){
    文;
    ・・・
    return 式;
}
```

複数の引数を指定するときは
引数の型 引数名を
コンマで区切って書く

return 式; は
戻り値がない場合(void型)は省略

- 例 Example 9-1

```
void rollDice(int numSides){
  int d = 1 + int(random(sumSides));
  println("Rolling... " + d);
}
```

戻り値の型	void
関数名	rollDice
引数の型	int
引数名	numSides

引数numSidesを使って
処理を行う

関数の基礎

- 1からnumSidesの整数の乱数を返す関数
  - void rollDice(int numSides)
  - 値の変換 P228
    - float型からint型に変換するため int()を使っている
      0から(numSides-1)の乱数 int(random(numSides))
    - 1からnumSidesの乱数にするため +1
      1 + int(random(numSides))

関数の基礎 Example 9-1・9-2

```
// Example 9-1
void setup() {
  println("Ready to roll!");
  rollDice(20);
  rollDice(20);
  rollDice(6);
  println("Finished.");
}

void rollDice(int numSides) {
  int d = 1 + int(random(numSides));
  println("Rolling... " + d);
}
```

関数 rollDice()を使う（呼ぶ）

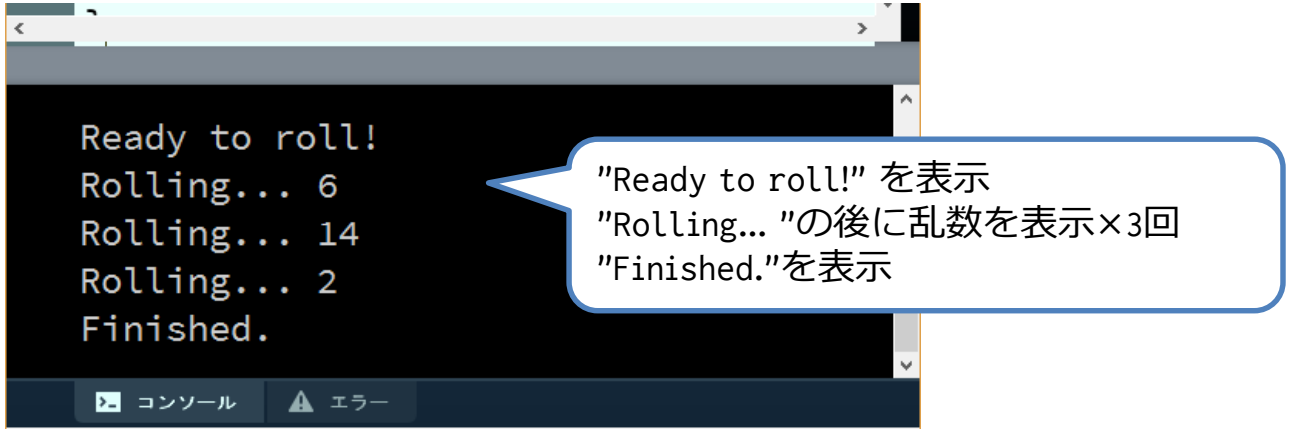
関数 rollDice()を作る（定義）
この関数は「サイコロを振る」という名前

```
// Example 9-2
void setup() {
  println("Ready to roll!");
  int d1 = 1 + int(random(20));
  println("Rolling... " + d1);
  int d2 = 1 + int(random(20));
  println("Rolling... " + d2);
  int d3 = 1 + int(random(6));
  println("Rolling... " + d3);
  println("Finished.");
}
```

共通する処理を3回
繰り返し実行している

共通する処理が複数回繰り返している
↓
共通部分を関数を使って
再利用できる形にまとめる

実行結果



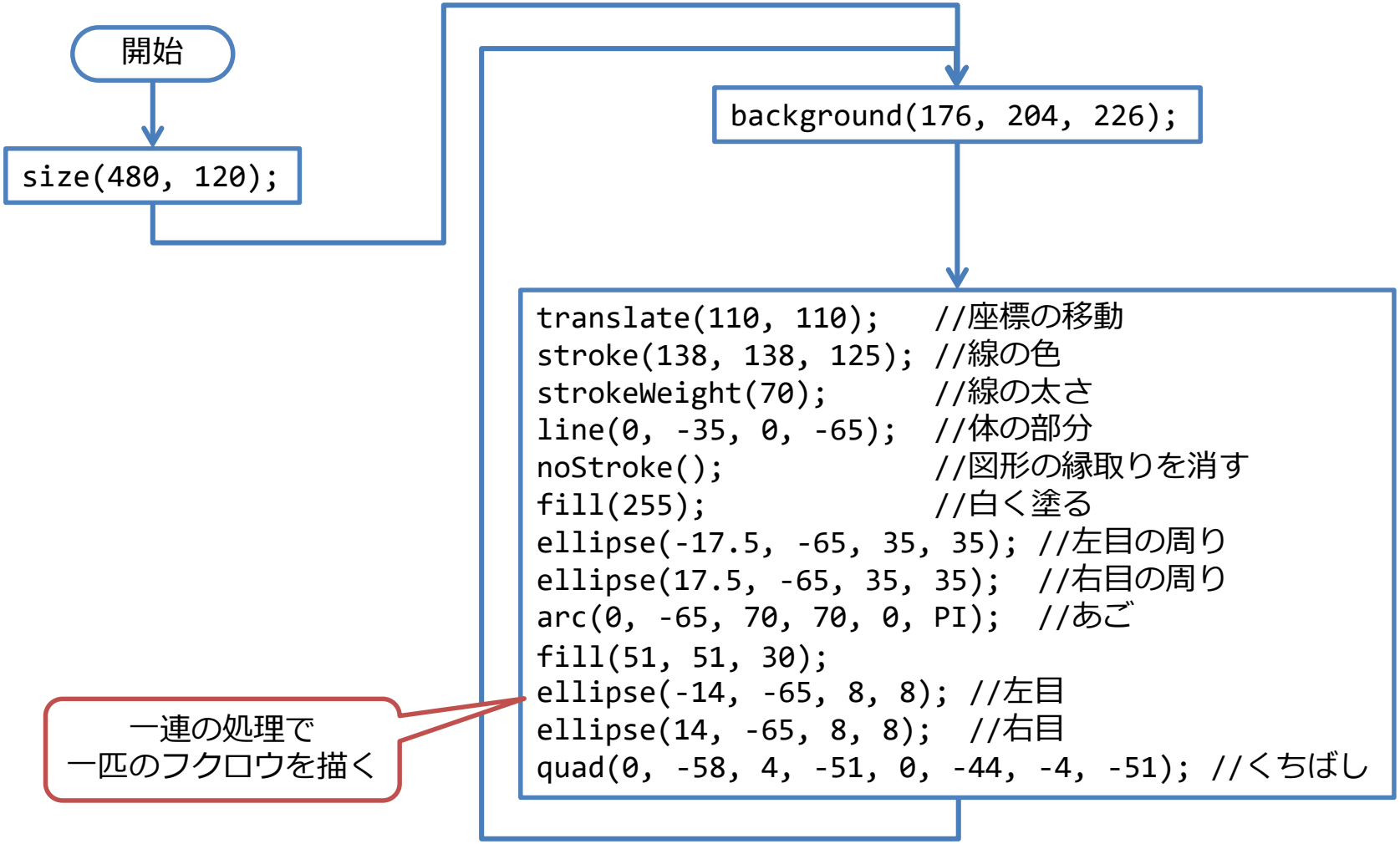
- 複数の命令をまとめて関数を作る
  - 処理内容が分かりやすく理解できる名前をつける
  - 同じような処理を何度も書かずに済む
  - 1度作った関数は何度も使える

関数を作る

- Example 9-3 フクロウを描く

```
void setup() {
  size(480, 120);
}

void draw() {
  background(176, 204, 226);
  translate(110, 110);
  stroke(138, 138, 125);
  strokeWeight(70);
  line(0, -35, 0, -65);
  noStroke();
  fill(255);
  ellipse(-17.5, -65, 35, 35);
  ellipse(17.5, -65, 35, 35);
  arc(0, -65, 70, 70, 0, PI);
  fill(51, 51, 30);
  ellipse(-14, -65, 8, 8);
  ellipse(14, -65, 8, 8);
  quad(0, -58, 4, -51, 0, -44, -4, -51);
}
```

フローチャート

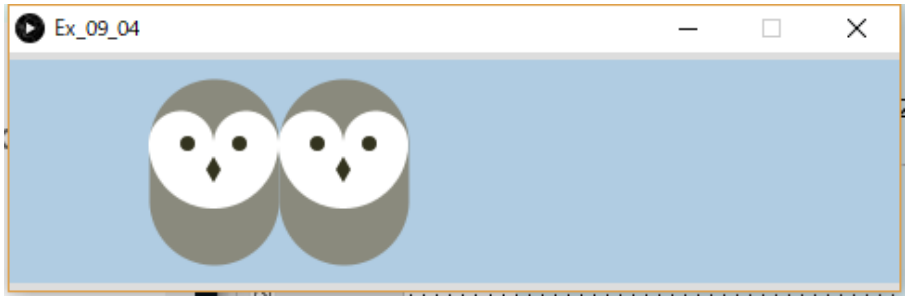

開始
size(480, 120);
background(176, 204, 226);
translate(110, 110); //座標の移動
stroke(138, 138, 125); //線の色
strokeWeight(70); //線の太さ
line(0, -35, 0, -65); //体の部分
noStroke(); //図形の縁取りを消す
fill(255); //白く塗る
ellipse(-17.5, -65, 35, 35); //左目の周り
ellipse(17.5, -65, 35, 35); //右目の周り
arc(0, -65, 70, 70, 0, PI); //あご
fill(51, 51, 30);
ellipse(-14, -65, 8, 8); //左目
ellipse(14, -65, 8, 8); //右目
quad(0, -58, 4, -51, 0, -44, -4, -51); //くちばし
一連の処理で
一匹のフクロウを描く

関数を作る

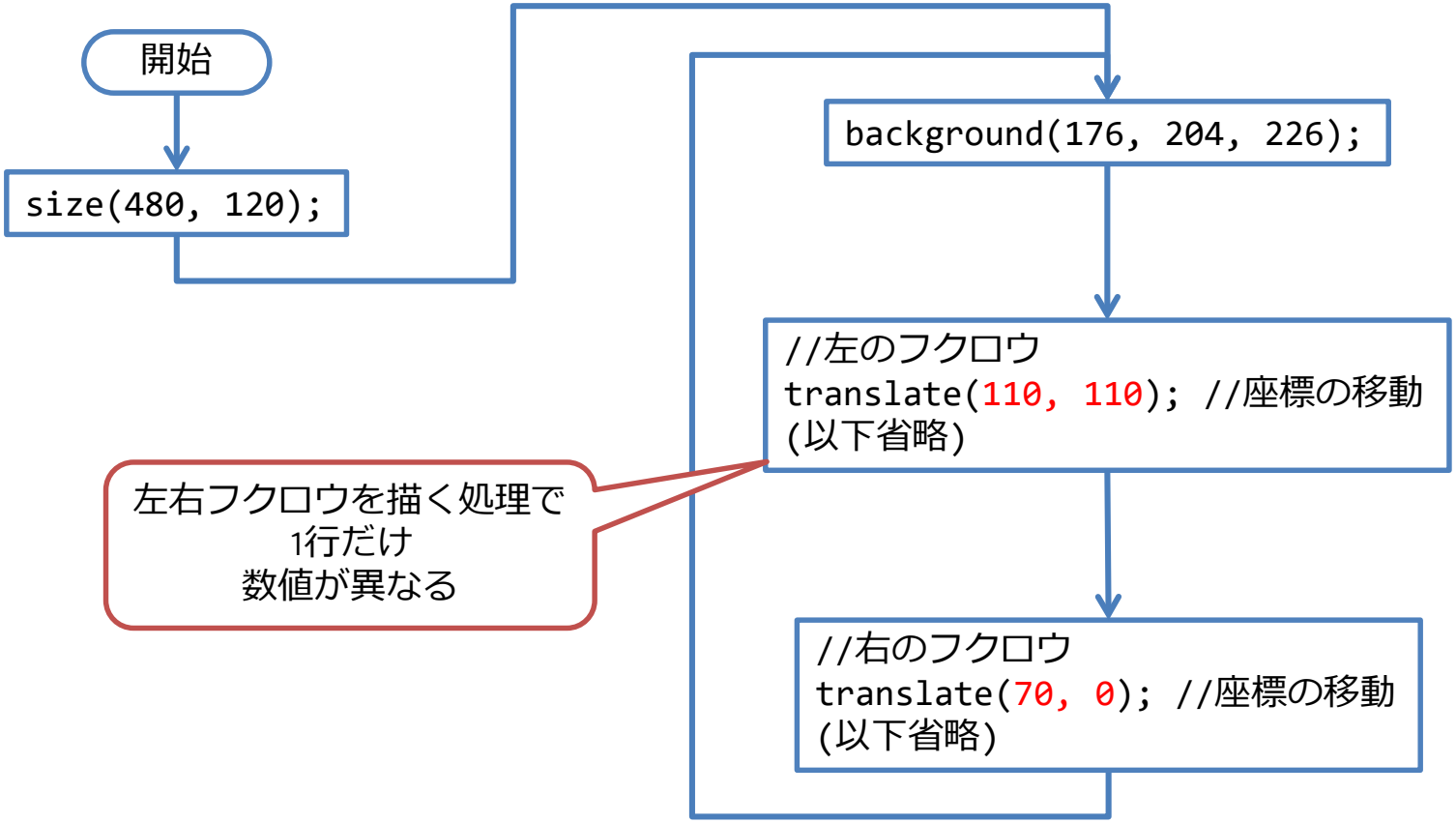
- Example 9-4 2羽のフクロウ

```
void setup() {
  size(480, 120);
}
void draw() {
  background(176, 204, 226);

  // 左のフクロウ
  translate(110, 110);
  stroke(138, 138, 125);
  strokeWeight(70);
  line(0, -35, 0, -65);
  noStroke();
  fill(255);
  ellipse(-17.5, -65, 35, 35);
  ellipse(17.5, -65, 35, 35);
  arc(0, -65, 70, 70, 0, PI);
  fill(51, 51, 30);
  ellipse(-14, -65, 8, 8);
  ellipse(14, -65, 8, 8);
  quad(0, -58, 4, -51, 0, -44, -4, -51);
```

```
  // 右のフクロウ
  translate(70, 0);
  stroke(138, 138, 125);
  strokeWeight(70);
  line(0, -35, 0, -65);
  noStroke();
  fill(255);
  ellipse(-17.5, -65, 35, 35);
  ellipse(17.5, -65, 35, 35);
  arc(0, -65, 70, 70, 0, PI);
  fill(51, 51, 30);
  ellipse(-14, -65, 8, 8);
  ellipse(14, -65, 8, 8);
  quad(0, -58, 4, -51, 0, -44, -4, -51);
}
```



フローチャート

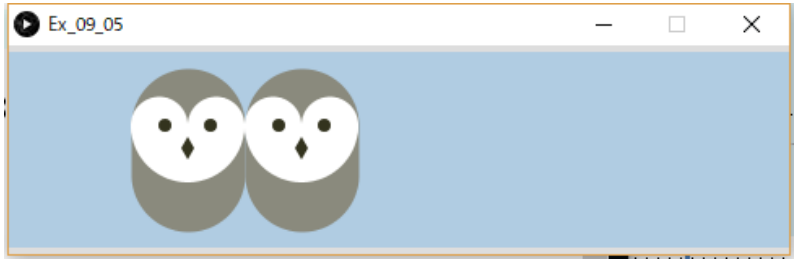

開始
size(480, 120);
background(176, 204, 226);
//左のフクロウ
translate(110, 110); //座標の移動
(以下省略)
左右フクロウを描く処理で
1行だけ
数値が異なる
//右のフクロウ
translate(70, 0); //座標の移動
(以下省略)

関数を作る

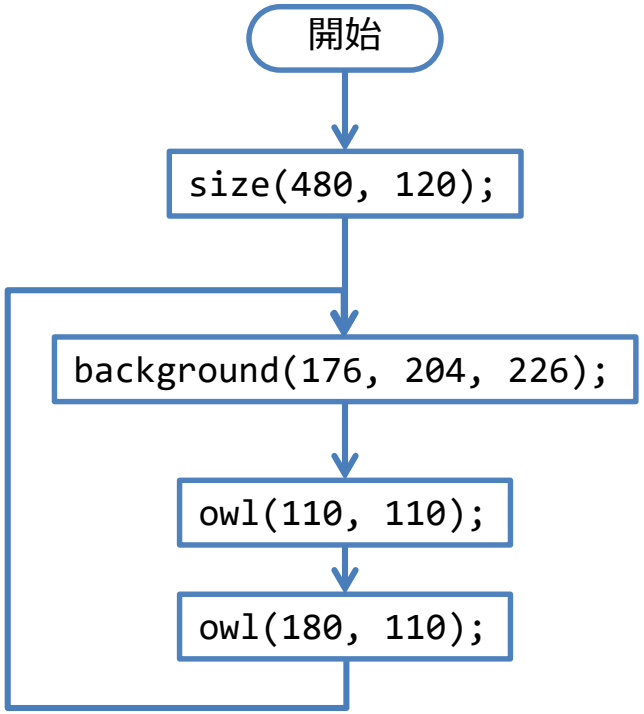
- Example 9-5 フクロウ関数

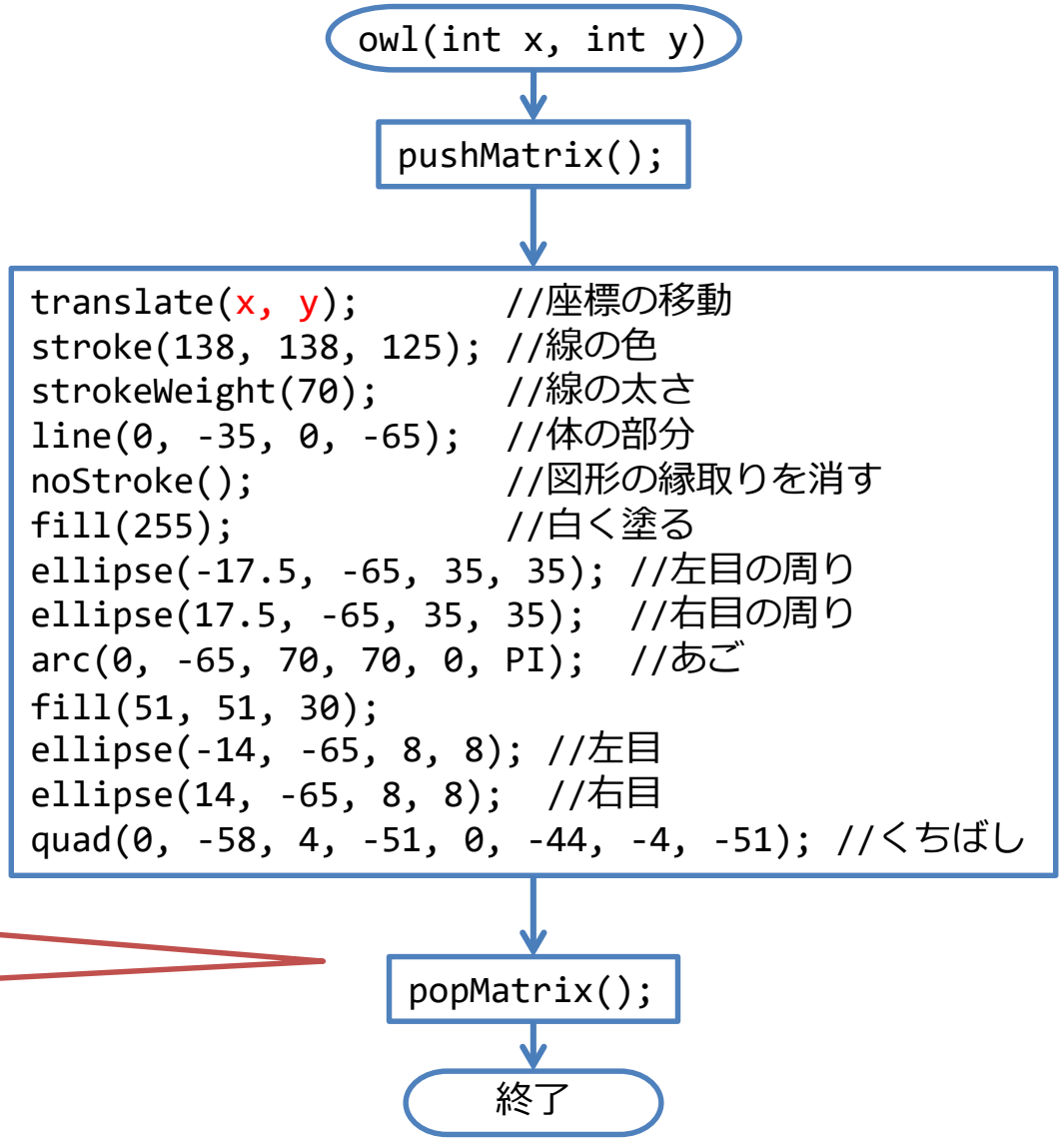
```
void setup() {
  size(480, 120);
}

void draw() {
  background(176, 204, 226);
  owl(110, 110);
  owl(180, 110);
}
```

```
void owl(int x, int y) {
  pushMatrix();
  translate(x, y);
  stroke(138, 138, 125);
  strokeWeight(70);
  line(0, -35, 0, -65);
  noStroke();
  fill(255);
  ellipse(-17.5, -65, 35, 35);
  ellipse(17.5, -65, 35, 35);
  arc(0, -65, 70, 70, 0, PI);
  fill(51, 51, 30);
  ellipse(-14, -65, 8, 8);
  ellipse(14, -65, 8, 8);
  quad(0, -58, 4, -51, 0, -44, -4, -51);
  popMatrix();
}
```



フローチャート





座標の移動・回転・伸縮などの
命令を使う場合に
処理の始めにpushMatrix()
処理の最後にpopMatrix()を追加

関数の定義

- 名前の付け方について
  - 英字・数字・アンダースコアなどを使える
    - ただし数字から始めない
  - 長さ制限なし
  - 予約語は使えない
  - 大文字・小文字は区別する

関数を呼び出す

- 関数の呼び出し方（使い方）

```
関数名(引数);
```

- 例 Example 8-1

```
void setup(){
  println("Ready to roll!");
  rollDice(20);
  rollDice(20);
  rollDice(6);
}
```

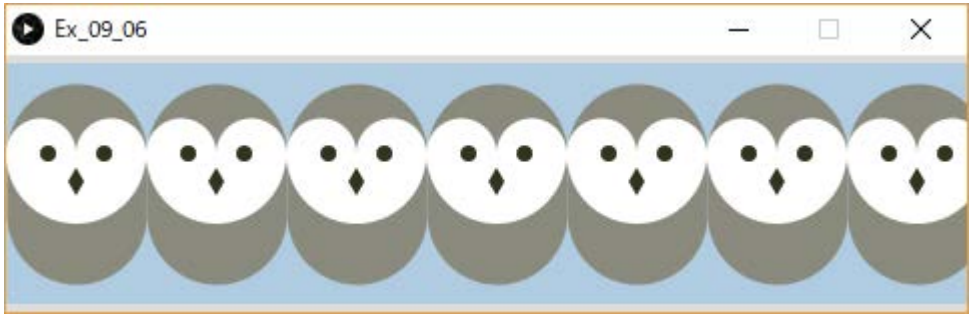
引数で値を渡して
関数を呼び出す

関数を作る

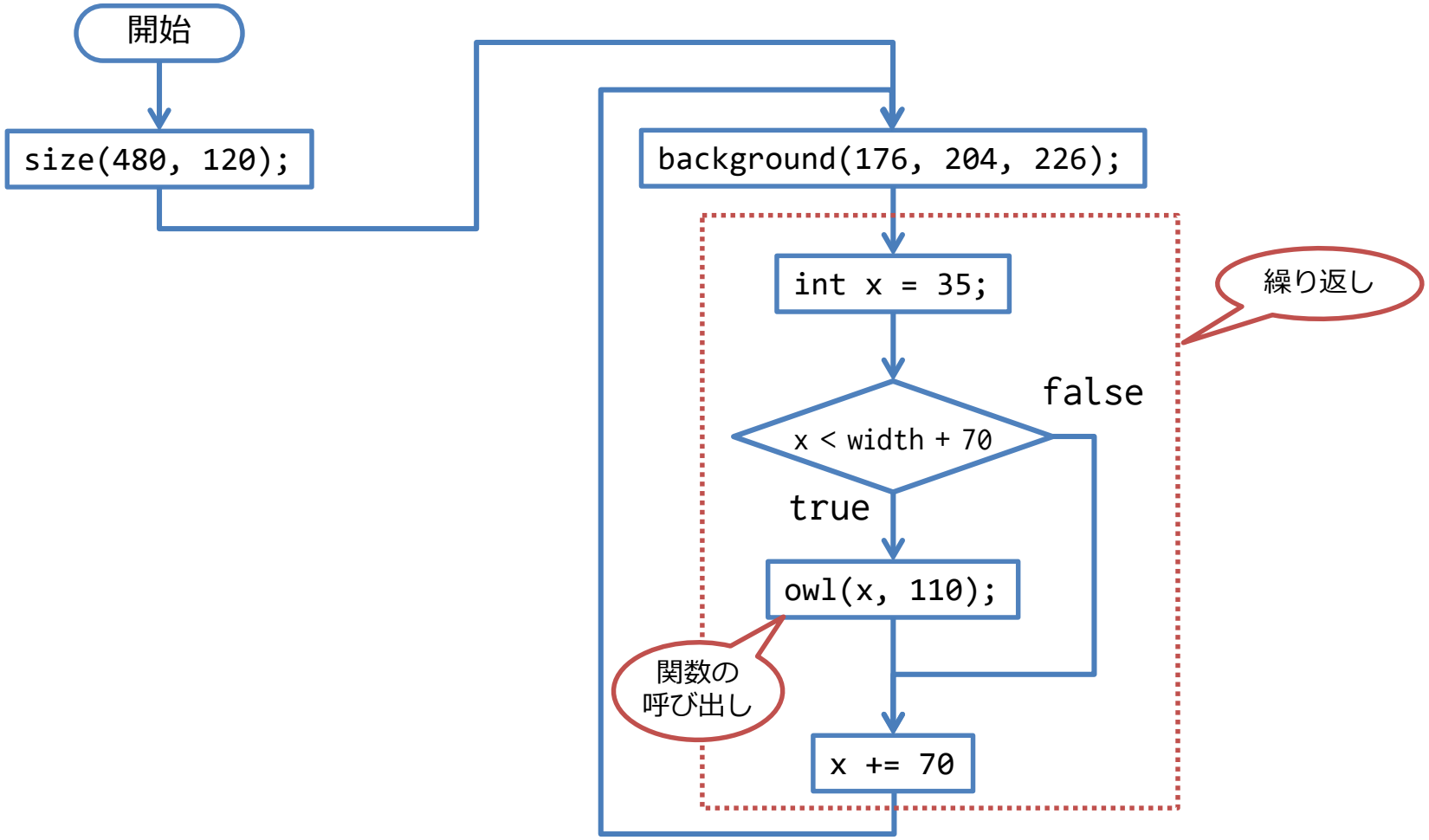
- Example 9-6 人口増加

```
void setup() {
  size(480, 120);
}

void draw() {
  background(176, 204, 226);
  for (int x = 35; x < width + 70; x += 70) {
    owl(x, 110);
  }
}
```

```
void owl(int x, int y) {

  //省略 Example 9-5 と同じ

}
```



フローチャート

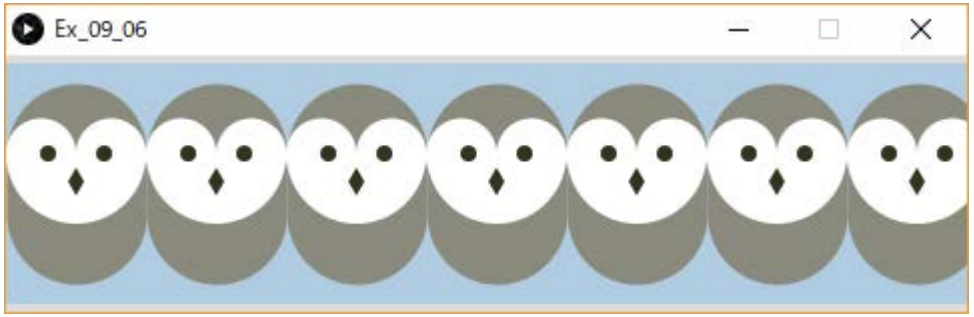

開始
size(480, 120);
background(176, 204, 226);
int x = 35;
x < width + 70
false
true
owl(x, 110);
関数の
呼び出し
x += 70
繰り返し

関数を作る

- Example 9-7 いろんな大きさのフクロウたち

```
void setup() {
  size(480, 120);
}

void draw() {
  background(176, 204, 226);
  for (int x = 35; x < width + 70; x += 70) {
    owl(x, 110);
  }
}
```

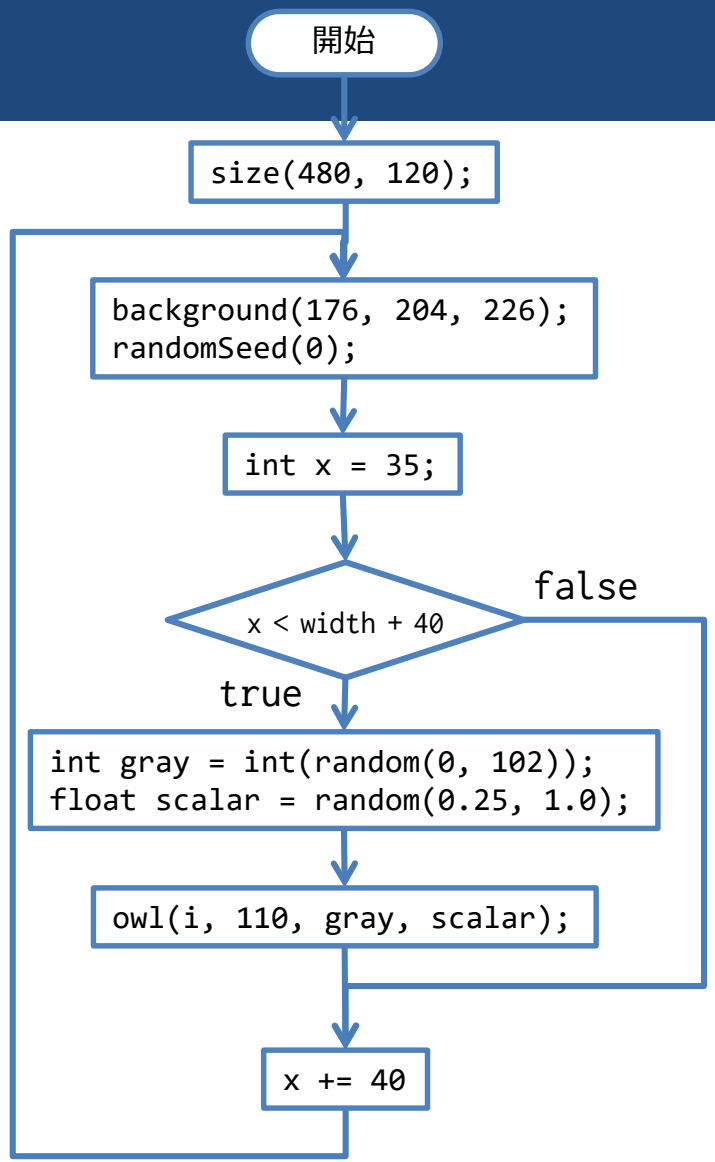
```
void owl(int x, int y) {

  //省略 Example 9-5 と同じ

}
```



関数を作る

- Example 9-7 いろんな大きさのフクロウたち

```
void setup() {
  size(480, 120);
}

void draw() {
  background(176, 204, 226);
  randomSeed(0);
  for (int i = 35; i < width + 40; i += 40) {
    int gray = int(random(0, 102));
    float scalar = random(0.25, 1.0);
    owl(i, 110, gray, scalar);
  }
}
```



関数を作る

```
void owl(int x, int y, int g, float s) {
  pushMatrix();
  translate(x, y);
  scale(s);
  stroke(138-g, 138-g, 125-g);
  strokeWeight(70);
  line(0, -35, 0, -65);
  noStroke();
  fill(255);
  ellipse(-17.5, -65, 35, 35);
  ellipse(17.5, -65, 35, 35);
  arc(0, -65, 70, 70, 0, PI);
  fill(51, 51, 30);
  ellipse(-14, -65, 8, 8);
  ellipse(14, -65, 8, 8);
  quad(0, -58, 4, -51, 0, -44, -4, -51);
  popMatrix();
}
```

実行する度
色・大きさが
変わる

```
owl(int x, int y, int g, float s)
```

```
pushMatrix();
```

```
translate(x, y);
scale(s);                          //追加
stroke(138-g, 138-g, 125-g);  //変更
strokeWeight(70);
line(0, -35, 0, -65);
noStroke();
fill(255);
ellipse(-17.5, -65, 35, 35);
ellipse(17.5, -65, 35, 35);
arc(0, -65, 70, 70, 0, PI);
fill(51, 51, 30);
ellipse(-14, -65, 8, 8);
ellipse(14, -65, 8, 8);
quad(0, -58, 4, -51, 0, -44, -4, -51);
popMatrix();
```

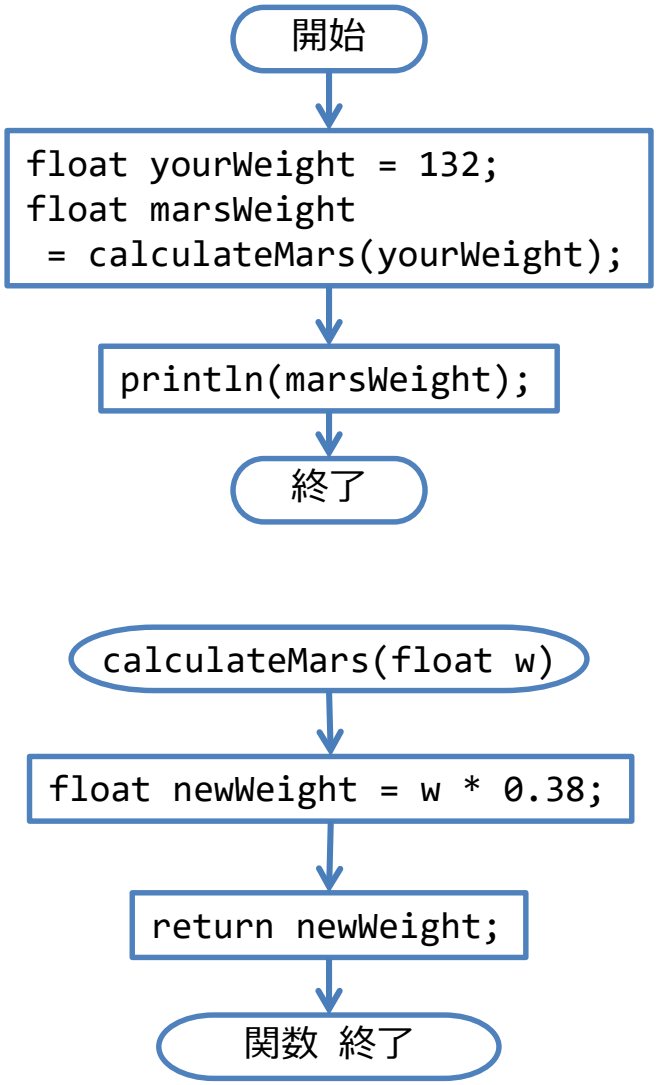
```
popMatrix();
```

関数 終了

値を返す

■ Example 9-8 値を返す

```
void setup() {
  float yourWeight = 132;
  float marsWeight = calculateMars(yourWeight);
  println(marsWeight);
}

float calculateMars(float w) {
  float newWeight = w * 0.38;
  return newWeight;
}
```

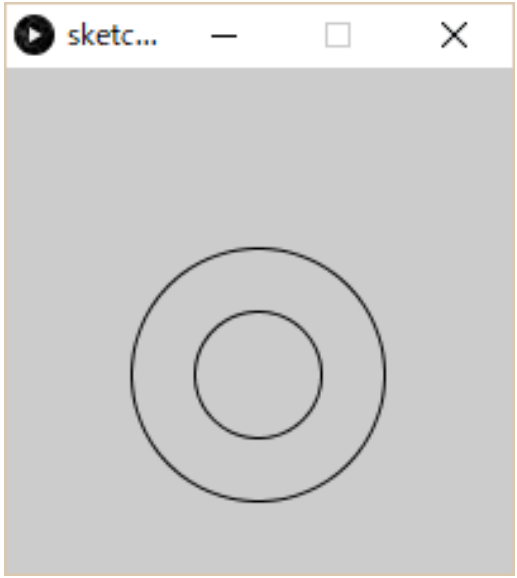


■ 値を返す

- return 式 ;

Kadai_09_01

- 二重丸を描く関数を作る
  - マウスの座標(mouseX, mouseY)を中心に、直径100と50の二重丸（内部は色を塗らない）を描くプログラムから、
    二重丸を描く関数
    doubleCircleを定義して
    関数を呼び出すように修正する



変更前 プログラム

```
void setup(){
  size(200, 200);
}

void draw(){
  background(204);

  noFill();                                    // 色を塗らない
  ellipse(mouseX, mouseY, 100, 100);      // 外側の円を描く
  ellipse(mouseX, mouseY, 50, 50);        // 内側の円を描く
}
```

変更後 プログラム

```
void setup(){
  size(200, 200);
}

void draw(){
  background(204);
    ?        X, mouseY);     // (mouseX, mouseY)に
                                        // 二重丸を描く
}
void doubleCircle(float x, float y){
    ?              // 色を塗らない
    ?              則の円(直径100)を描く
    ?              則の円(直径50)を描く
}
```

Kadai_09_02

- Kadai_09_01の二重丸を描く関数を修正し、直径dを引数に追加して、二重丸の外側の円の大きさを指定できるようにする

プログラム

```
void setup(){
  size(200, 200);
}

void draw(){
  background(204);
    ?        X, mouseY, 100);// (mouseX, mouseY)に
                                          // 直径100の二重丸を描く
}
void doubleCircle(        ?
    ?              // 色を塗らない
    ?              則の円(直径d)を描く
    ?              則の円(直径d/2)を描く
}
```

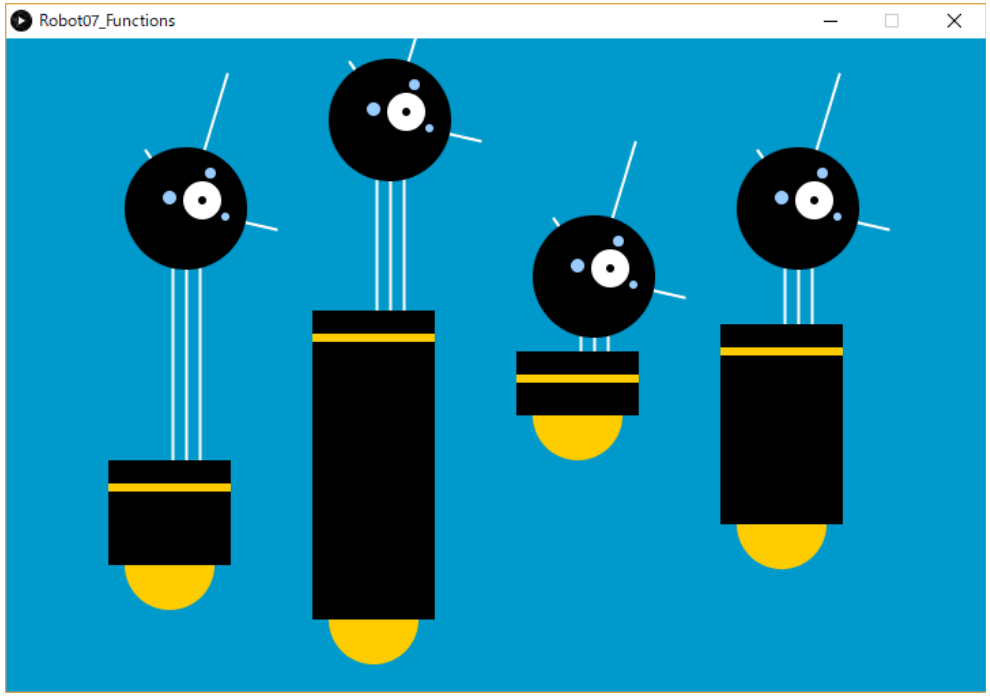
Robot 7: Functions

```
void setup() {
  size(720, 480);
  strokeWeight(2);
  ellipseMode(RADIUS);
}

void draw() {
  background(0, 153, 204);
  drawRobot(120, 420, 110, 140);
  drawRobot(270, 460, 260, 95);
  drawRobot(420, 310, 80, 10);
  drawRobot(570, 390, 180, 40);
}
```

ロボットを描く関数を
4回呼び出す

ロボットを描く関数
drawRobot()
ロボットの位置(x,y)
胴体の長さ bodyHeight
首の長さ neckHeight
を引数として指定する

```
void drawRobot(int x, int y, int bodyHeight, int neckHeight) {

  int radius = 45;
  int ny = y - bodyHeight - neckHeight - radius;

  // Neck
  stroke(255);
  line(x+2, y-bodyHeight, x+2, ny);
  line(x+12, y-bodyHeight, x+12, ny);
  line(x+22, y-bodyHeight, x+22, ny);

  // Antennae
  line(x+12, ny, x-18, ny-43);
  line(x+12, ny, x+42, ny-99);
  line(x+12, ny, x+78, ny+15);
```

Robot 7: Functions

```
  // Body
  noStroke();
  fill(255, 204, 0);
  ellipse(x, y-33, 33, 33);
  fill(0);
  rect(x-45, y-bodyHeight, 90, bodyHeight-33);
  fill(255, 204, 0);
  rect(x-45, y-bodyHeight+17, 90, 6);

  // Head
  fill(0);
  ellipse(x+12, ny, radius, radius);
  fill(255);
  ellipse(x+24, ny-6, 14, 14);
  fill(0);
  ellipse(x+24, ny-6, 3, 3);
  fill(153, 204, 255);
  ellipse(x, ny-8, 5, 5);
  ellipse(x+30, ny-26, 4, 4);
  ellipse(x+41, ny+6, 3, 3);
}
```